

Last time: Pseudorandom Function (PRF)

A PRF is a function $F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ such that for any PPT distinguishers D , there is a negligible function negl such that

$$|\Pr[D^{F_k(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

For uniformly random $k \in \{0,1\}^n$ and a random function $f \in \text{Func}_n$

Pseudorandom Function (PRF)

- A random function instead of a random string in PRG
- $F: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n \quad F(k, x) \rightarrow y$
- Or $F_k: \{0,1\}^n \rightarrow \{0,1\}^n \quad F_k(x) \rightarrow y$
- An efficient attacker cannot differentiate it from a random function

Number of functions

- How many functions with n bits of input and output? $F: \{0,1\}^n \rightarrow \{0,1\}^n$
- A big lookup table with 2^n entries
 1. 2^n possibilities each, so $(2^n)^{2^n} = 2^{n \cdot 2^n}$
 2. 2^n strings of size n . $n \cdot 2^n$ bits in total to represent a table. $2^{n \cdot 2^n}$ different tables.

Definition of PRF

A PRF is a function $F : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ such that for any PPT distinguishers D , there is a negligible function negl such that

$$|\Pr[D^{F_k(\cdot)}(1^n) = 1] - \Pr[D^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(n)$$

For uniformly random $k \in \{0,1\}^n$ and a random function $f \in \text{Func}_n$

PRF vs PRG

- $PRG : \{0,1\}^n \rightarrow \{0,1\}^{2n}$
 - Key space size: 2^n # of random strings: 2^{2n}
- $PRF : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$
 - Key space size: 2^n # of random functions : $2^{n \cdot 2^n}$

PRF vs PRG

PRF F immediately implies a PRG G :

Define $\text{PRG}(k) = F_k(1) \mid F_k(2) \mid F_k(3) \mid \dots$

PRF



PRG



Goldreich-Goldwasser-Micali construction

CPA-security from PRF

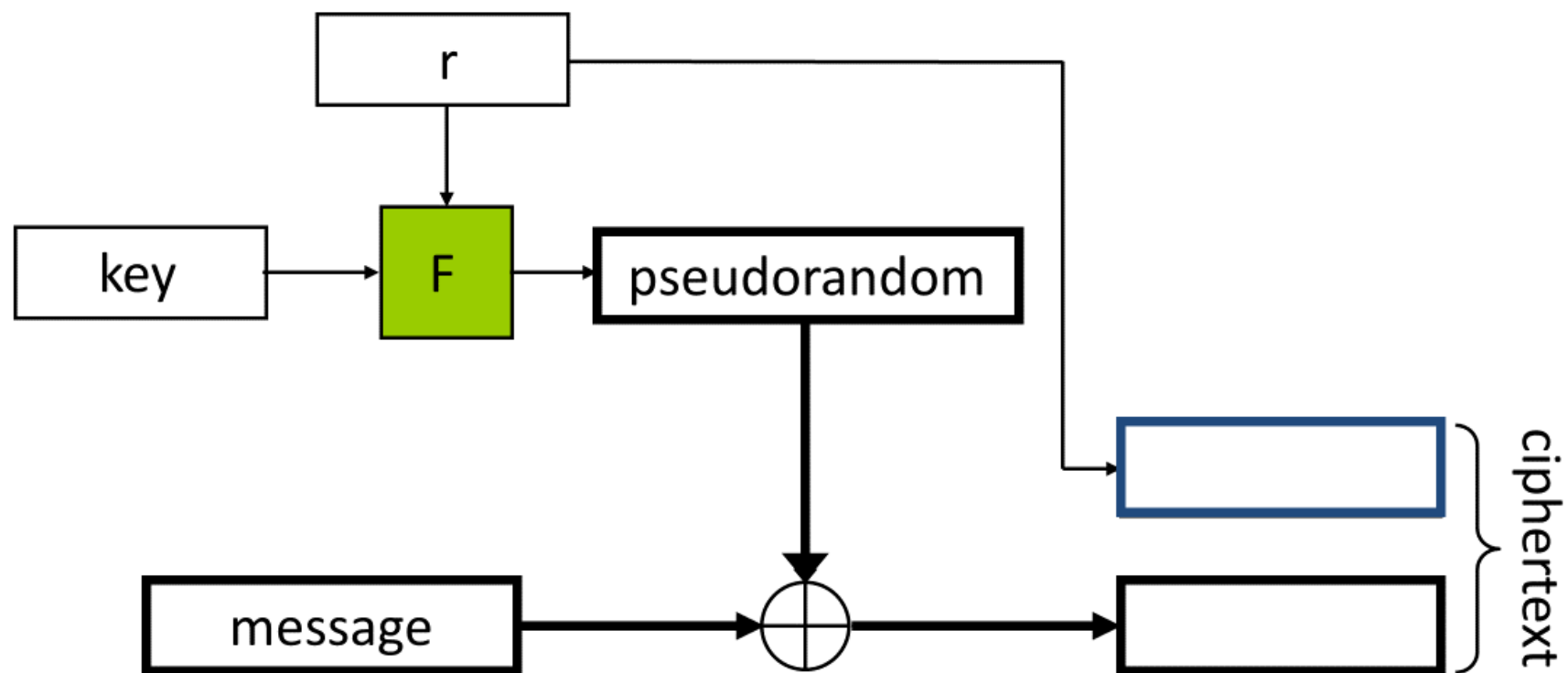
Construction:

- $\text{Gen}(1^n)$: choose uniform $k \in \{0,1\}^n$ and output it as the key
- $\text{Enc}_k(m)$: on input a message $m \in \{0,1\}^n$, sample uniform $r \in \{0,1\}^n$ and compute

$$c = (r, F_k(r) \oplus m)$$

- $\text{Dec}_k(c)$: on input a ciphertext $c = (r, s)$, compute

$$m = F_k(r) \oplus s$$



Chosen-plaintext attack (CPA) security

Experiment $\text{PrivK}_{A,\Pi}^{\text{CPA}}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** . A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_k(m_b)$ is given to A .
4. A continues to have **oracle access to $\text{Enc}_k(\cdot)$** and outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Definition: a private-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is CPA-secure, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{PrivK}_{A,\Pi}^{\text{CPA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

Proof idea

- Reduction, build a **distinguisher** for PRF
- Build the big lookup table for function f . Give random output for the random function f , and $F_k(r)$ for PRF

Two cases: (1) r has been queried by the adversary $\rightarrow$ some gain of success; (2) r has not been queried by the adversary $\rightarrow$ negl from PRF

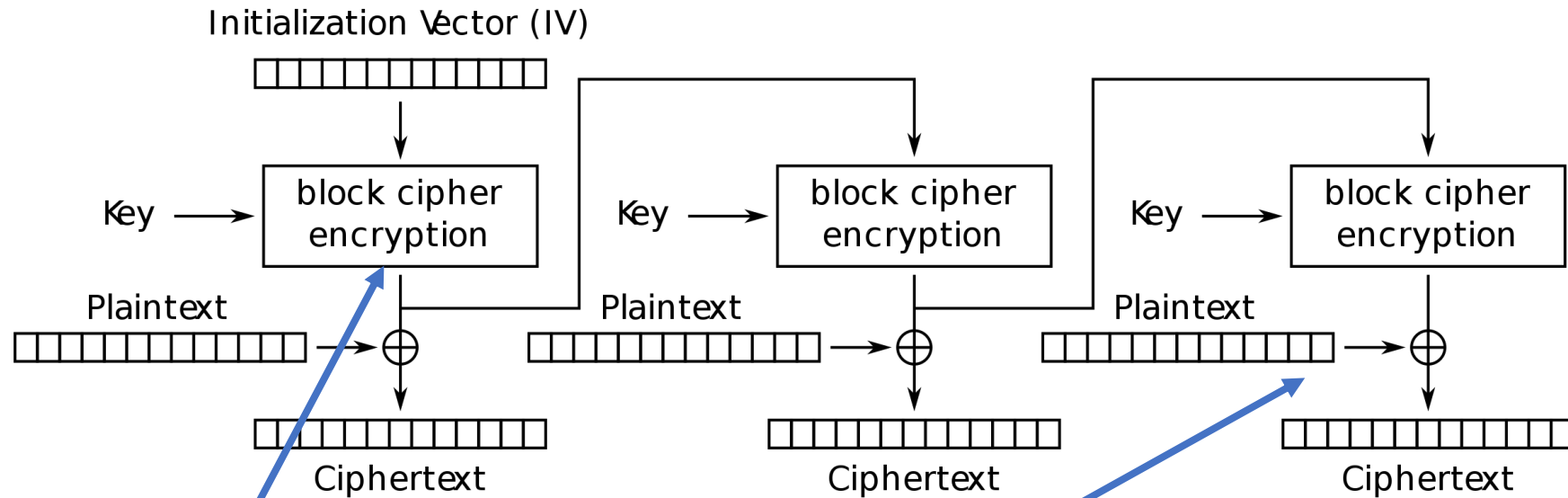
Total probability: $\leq \frac{1}{2} + \frac{q(n)}{2^n} + \text{negl}(n)$

Pseudorandom Permutations

- One-to-one, a subset of PRF
- Invertible. Both $F_k(x) \rightarrow y$ and $F_k^{-1}(y) \rightarrow x$ are efficiently computable.

$$\left| \Pr \left[D^{F_k(\cdot), F_k^{-1}(\cdot)}(1^n) = 1 \right] - \Pr \left[D^{f(\cdot), f^{-1}(\cdot)}(1^n) = 1 \right] \right| \leq \text{negl}(n)$$

Output Feedback (OFB) mode



Output Feedback (OFB) mode encryption

1. Block cipher
2. Mode of encryption
3. Padding

Designing block ciphers

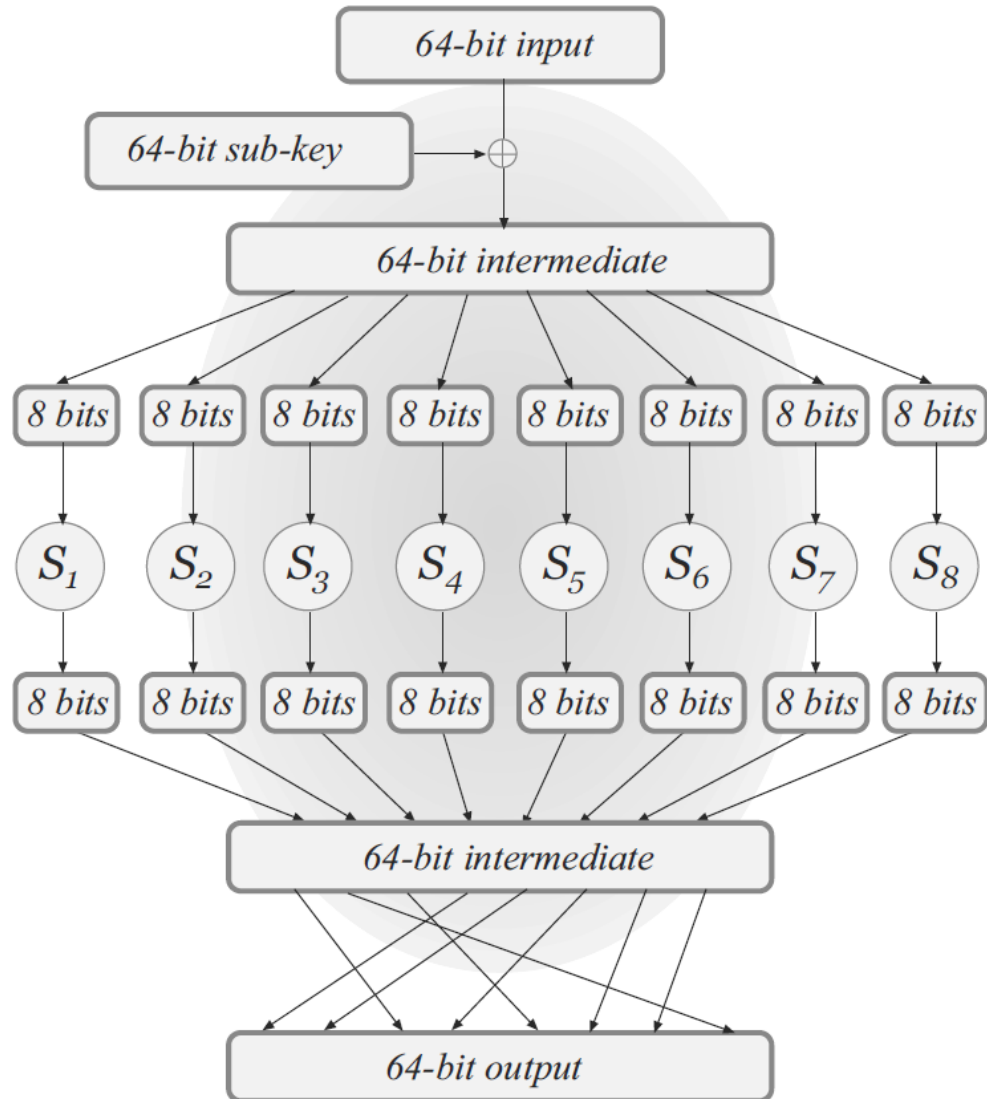
- Want F_k (for uniform, unknown key k) to be indistinguishable from a random function
- If x and x' differ in one bit, what should the relation between $F_k(x)$ and $F_k(x')$ be?
 - How many bits should change (on average)?
 - Which bits should change?

Confusion-diffusion paradigm

$$F_k(x) = f_1(x_1) || f_2(x_2) || \dots || f_{16}(x_{16})$$

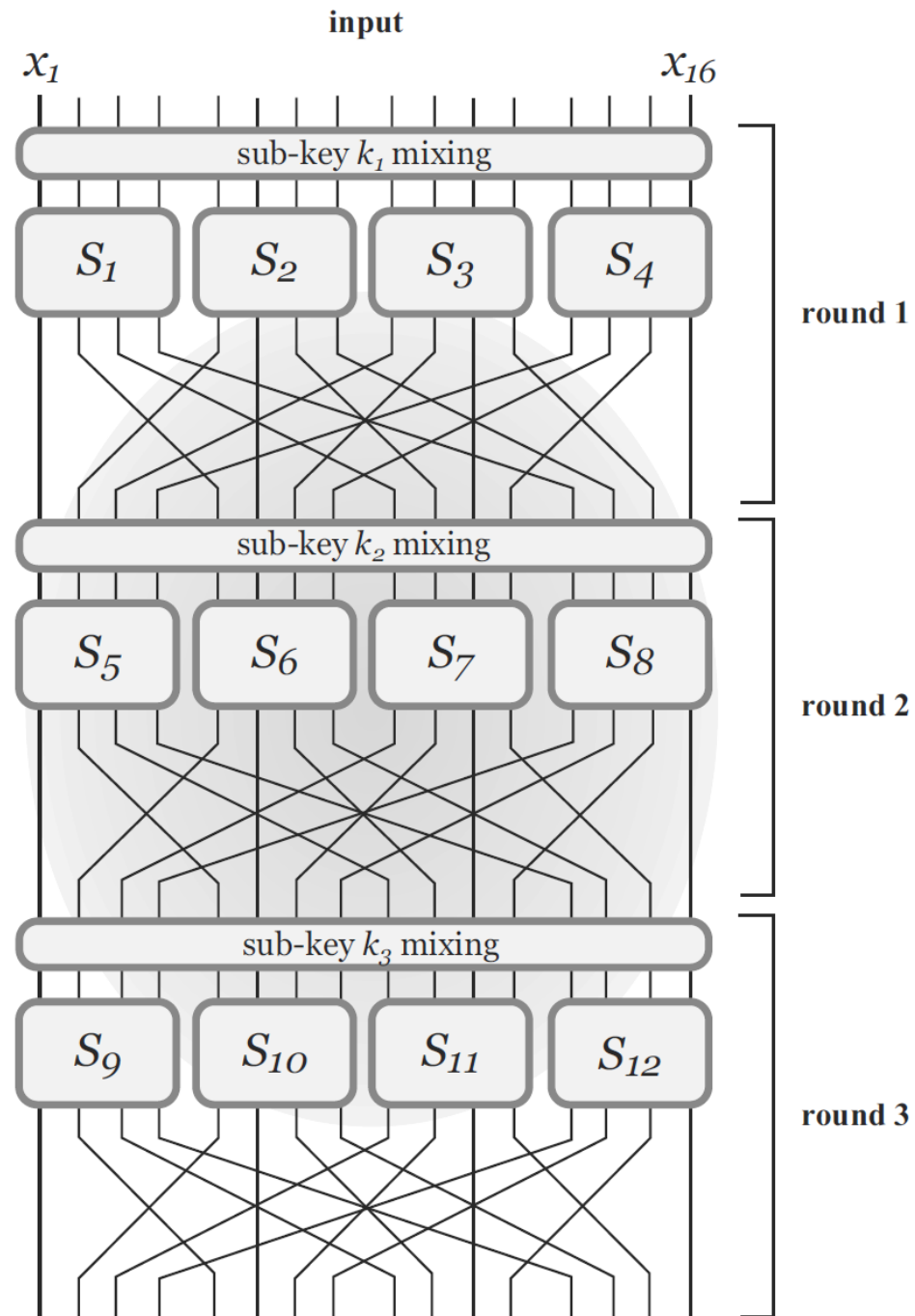
- Smaller random blocks $f_i()$: confusion
- Mixing permutation: diffusion

Substitution-Permutation Networks (SPN)



1. XOR with secret key
2. Substitution boxes
3. Permutation

Multi-round design



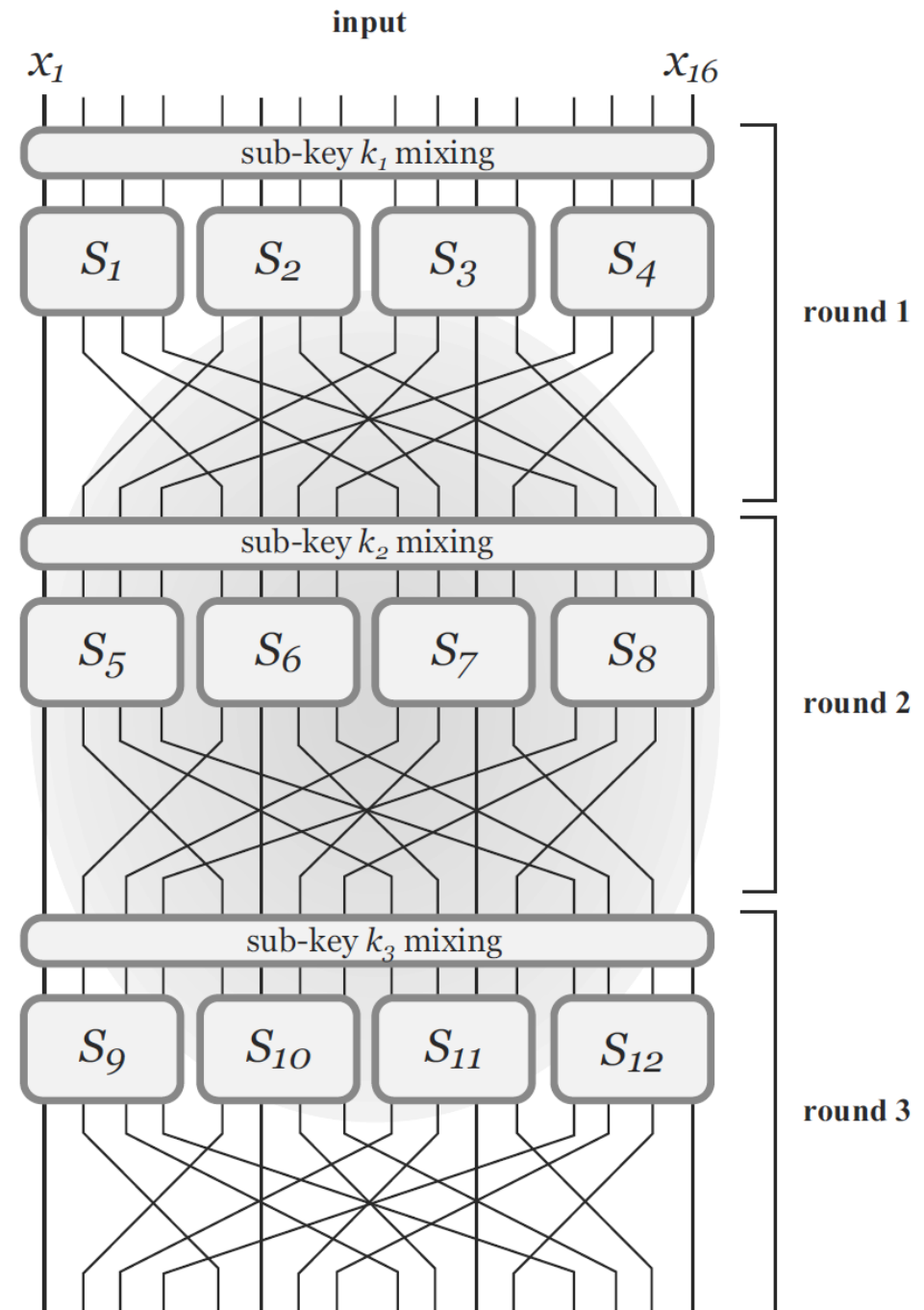
Avalanche effect

1-bit difference in the input should “affect” every bit of the output.

Avalanche effect

- Design S-boxes and mixing permutation to ensure *avalanche effect*
 - Small differences should eventually propagate to entire output
- S-boxes: *any* 1-bit change in input causes ≥ 2 -bit change in output, **non-linear!**
 - Not so easy to ensure!
- Mixing permutation
 - Each bit output from a given S-box should feed into a *different* S-box in the next round

Avalanche effect



Sub-keys and master key

- Master key: secret key
- Sub-keys for different rounds are derived from the master key
- $L+1$ sub-keys for L -round SPN: always XOR a sub-key at the end

AES

- AES-128, AES-192, AES-256: different key lengths. A block is 128-bit
- A 128-bit sub-key is derived from the master key. 4×4 bytes

https://en.wikipedia.org/wiki/AES_key_schedule

AES substitution boxes

$\{0,1\}^8 \rightarrow \{0,1\}^8$

AES S-box

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Multi-round

- AES has 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys.

Open design principle

Security should only rely on the secret key

Assumptions

AES/DES is a PRF/PRP

What's the difference? Enigma machine, Vigenère cipher

- Modular and composable constructions
- Years of cryptanalysis

Don't Roll Your Own Crypto!!!

National Security

Johns Hopkins researchers poke a hole in Apple's encryption



A group of Johns Hopkins University researchers found a bug in Apple's encryption that would let a skilled attacker decrypt photos and videos that were sent as secure instant messages. (Matthias Schrader/AP)

By [Ellen Nakashima](#)

March 21, 2016

Most Read World

1 Philippines on high alert as 'explosive' Super Typhoon Noru makes landfall



2 Italy poised for glass ceiling-shattering vote, hard right turn



3 Russian mobilization blasted for rounding up sick, disabled and elderly



4 As Ukraine's fathers fight, mothers go it alone with their children



5 Ukraine live briefing: Protesters scuffle with Russian police as mobilization inflames tensions



Latest episode

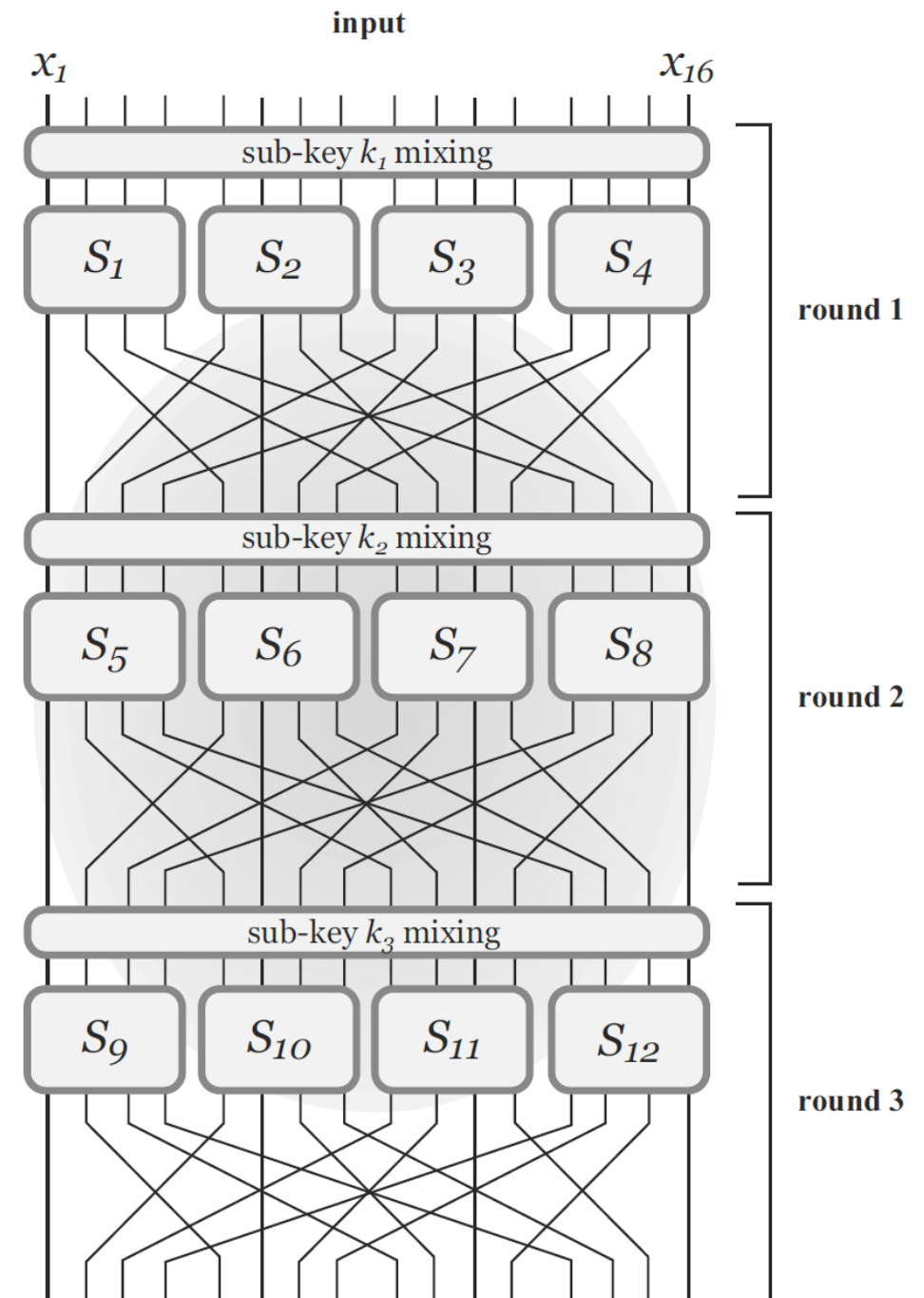
Why Russians have had enough with this war

► Listen 22:22

Unparalleled reporting Expert insight Clear

AES is invertible given k

- S-box is invertible
- We do need AES decryptions in some block cipher modes



Permutations vs functions

- Permutation: one-to-one function $F: \{0,1\}^n \rightarrow \{0,1\}^n$
- How many one-to-one functions with n bits of input and output?
- $(2^n)!$
- Stirling's approximation: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$
- $(2^n)! \approx \left(\frac{2^n}{e}\right)^{2^n}$

Pseudorandom Permutations

- One-to-one, a subset of PRF
- Invertible. Both $F_k(x) \rightarrow y$ and $F_k^{-1}(y) \rightarrow x$ are efficiently computable.

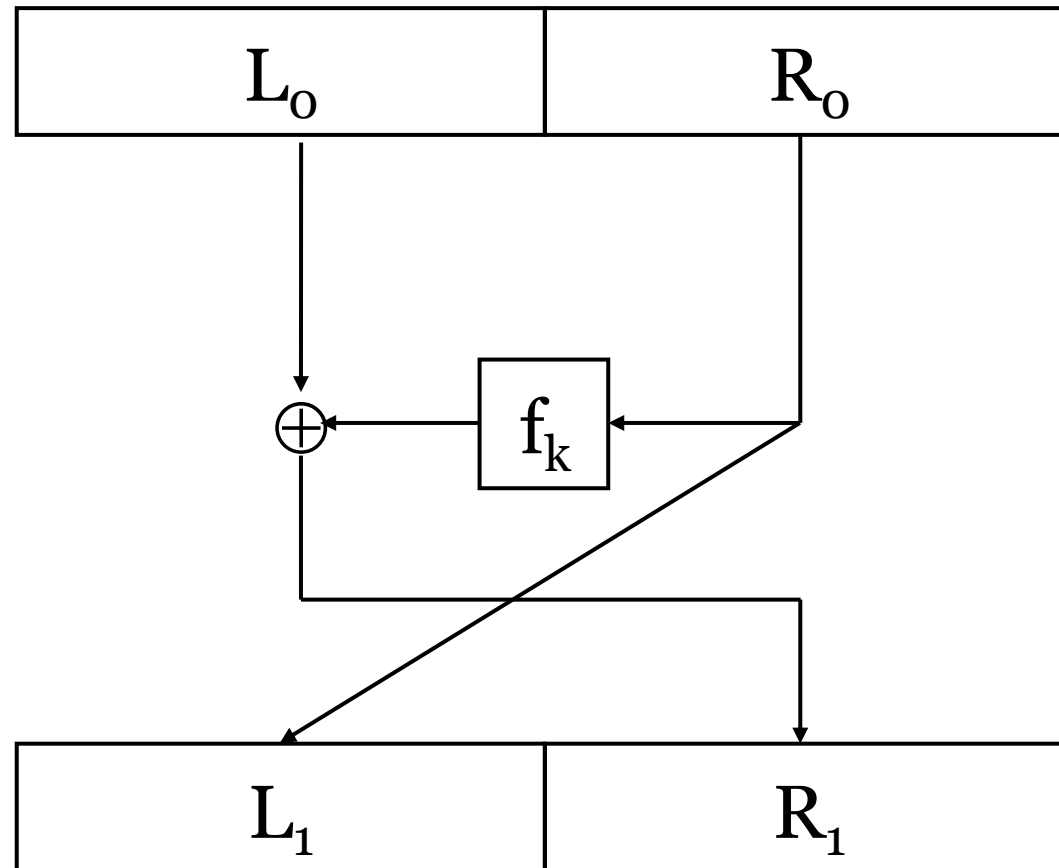
$$\left| \Pr \left[D^{F_k(\cdot), F_k^{-1}(\cdot)}(1^n) = 1 \right] - \Pr \left[D^{f(\cdot), f^{-1}(\cdot)}(1^n) = 1 \right] \right| \leq \text{negl}(n)$$

Feistel network

- Used in DES
- Advantage over SPN: S-boxes don't need to be invertible

Feistel network

$$L_i = R_{i-1}, R_i = L_{i-1} \oplus f_i(R_{i-1})$$



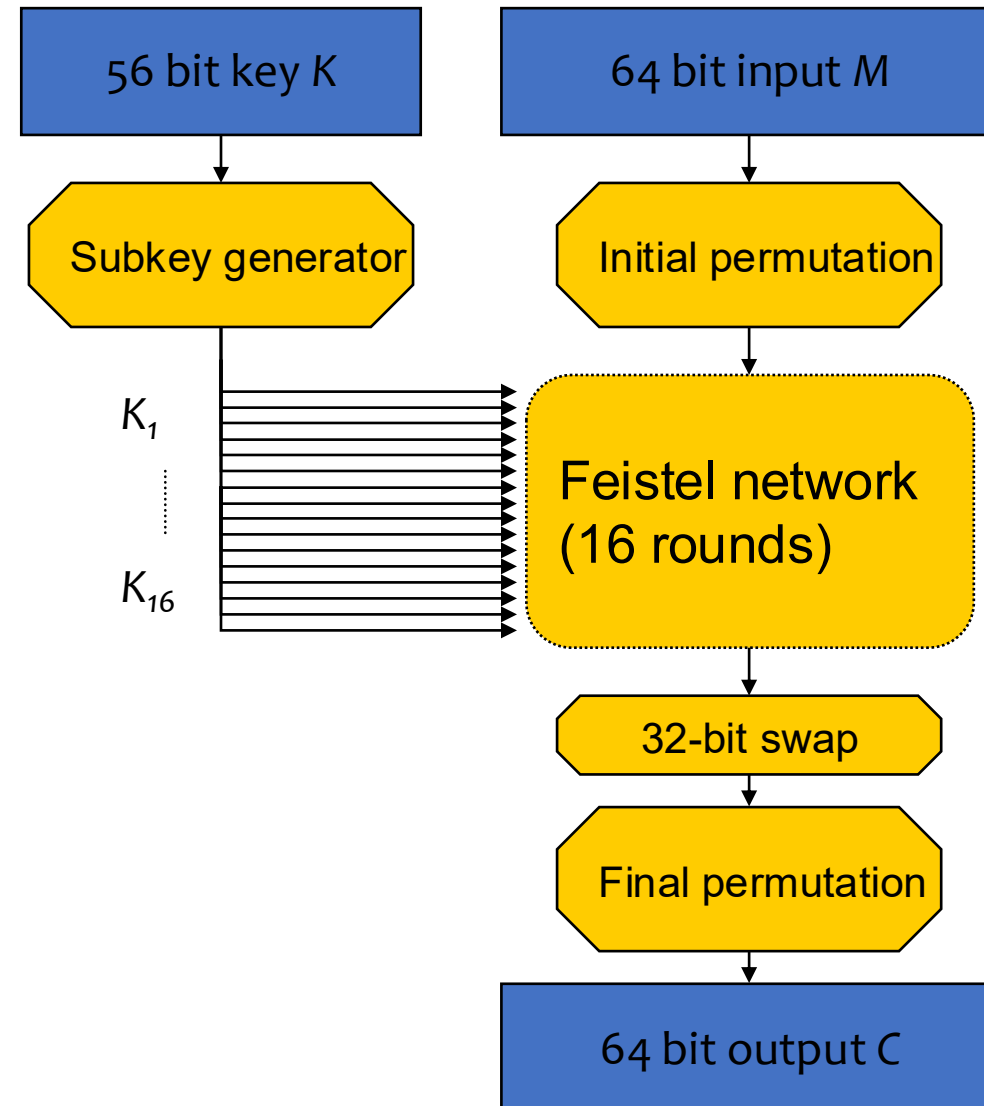
Inverting Feistel network

$$L_i = R_{i-1}, R_i = L_{i-1} \oplus f_i(R_{i-1})$$

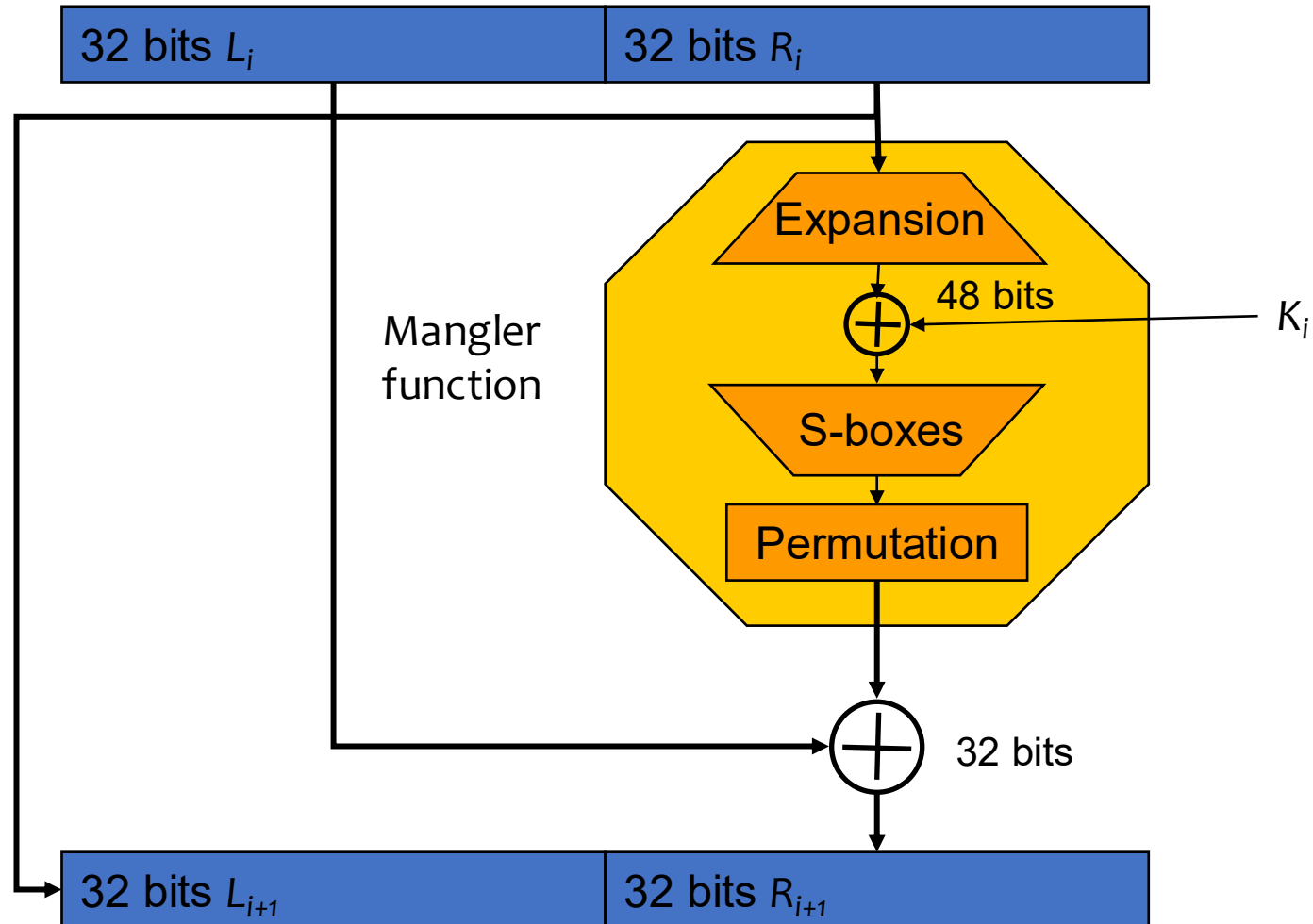
Given L_i, R_i , $R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus f_i(R_{i-1})$

DES overview

- Developed in 1970s
- Block length 64 bits, key length 56 bits
- Feistel cipher, 16 stages
- $K_1, \dots, K_{16}$ are 48 bit subkeys derived from K



Feistel network in DES



Substitution Boxes in DES

- 8 different S-Boxes (48 bits input)
- 6-bit input to 4-bit output (48 to 32 bits)
- Not invertible

DES mangler function

- S-boxes
 - Changing 1 bit of input changes at least 2 bits of output
- Mixing permutation
 - The 4 bits of output from any S-box affect the input to 6 S-boxes in the next round

Avalanche effect

- Consider 1-bit difference in left half of input
 - After 1 round, 1-bit difference in right half
 - S-boxes cause a 2-bit difference, implying a 3-bit difference overall after 2 rounds
 - Mixing permutation spreads differences into different S-boxes
 - ...

Security of DES

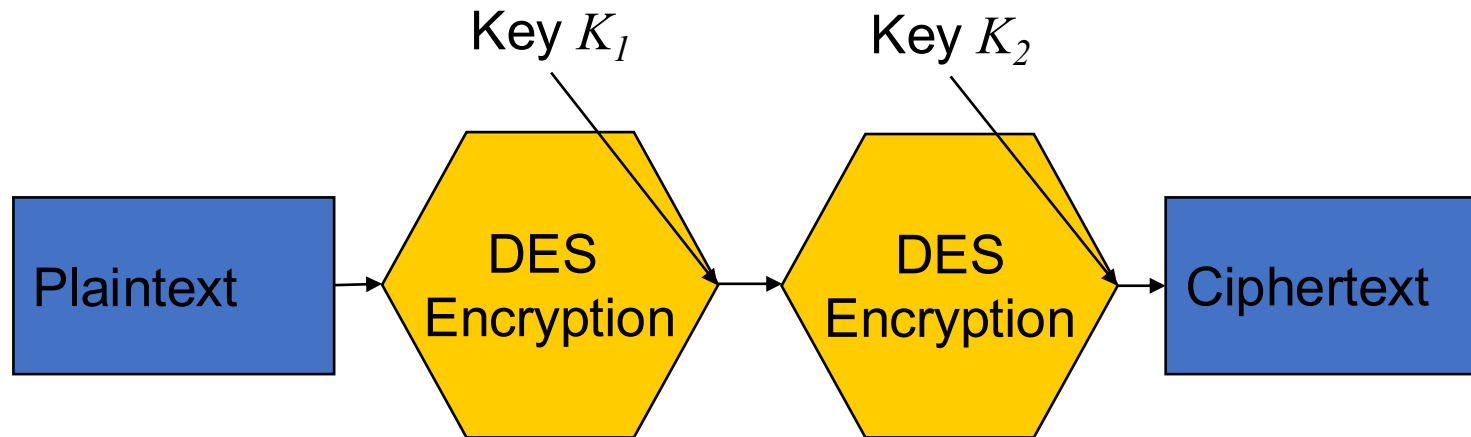
- DES is extremely well-designed
 - Except for some attacks that require large amounts of plaintext, no attacks better than brute-force are known
- But ... parameters are too small!

56-bit key length

- A concern as soon as DES was released
- Brute-force search over 2^{56} keys is possible
 - 1997: 1000s of computers, 96 days
 - 1998: distributed.net, 41 days
 - 1999: Deep Crack (\$250,000), 56 hours
 - Today: 48 FPGAs, ~1 day

Double DES

- Simple extension to DES, using two 56-bit keys

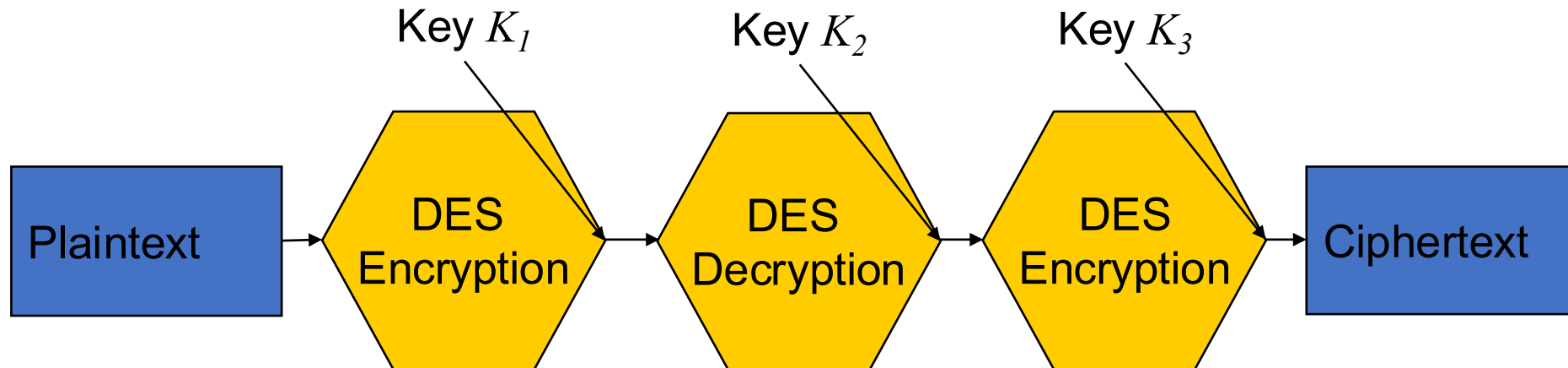


Meet-in-the-middle attack

- No! There is an attack taking 2^n time...
 - (And 2^n memory)
- The attack applies any time a block cipher can be “factored” into 2 independent components

Triple DES

- Simple extension to DES, using or three 56-bit keys



- Security level of 2^{112} , currently thought to be enough
 - Can also use K_1, K_2, K_1